



PARTITIONING & SHUFFLE

The Hidden Cost Behind Every groupBy, join & repartition

Partitions

Shuffle

Skew

Coalesce

Interview Ready in 6 Slides

Swipe to start learning →

P PARTITIONS — THE UNIT OF DISTRIBUTION

A partition is a chunk of a DataFrame that lives on **one Executor** and is processed by exactly **one Task**.

Where Partition Count Comes From

1 Reading a file — Spark sizes partitions off the file's block splits



2 `spark.sql.files.maxPartitionBytes` caps the size of each input partition



3 A wide transform (**groupBy**, **join**) repartitions via `spark.sql.shuffle.partitions` (default 200)



4 `.repartition(n)` / `.coalesce(n)` manually override the partition count

Interview Tip: Partition count sets your ceiling on parallelism — a Stage can never run more concurrent Tasks than it has partitions.

S DATA CROSSING THE WIRE

OPERATIONS THAT FORCE A SHUFFLE

`.groupBy()`

`.join()`

`.distinct()`

`.repartition()`

`.orderBy()`

Why Shuffles Are So Expensive

1 Map side writes shuffle data to **local disk** (shuffle write)



2 Data transfers across the **network** to the reduce side



3 Reduce side reads it back and **re-sorts / re-groups** it (shuffle read)



4 Combined disk + network I/O makes shuffle the **single biggest cost** in most jobs

Interview Tip: If a job is slow, check Shuffle Read/Write in the Spark UI before touching anything else.

K SKEW — THE SILENT STAGE-KILLER

Skew happens when a few partition keys hold far more data than the rest — one Task runs long after every other Task has finished.

Spotting and Fixing Skew

1

Symptom: Spark UI shows 199 Tasks done in 10s, 1 Task still running 20 minutes later



2

Cause: a hot key (NULL, a popular customer_id) dominates one partition



3

Fix — Salting: append a random suffix to the skewed key before the shuffle



4

Fix — AQE: Adaptive Query Execution can auto-split skewed partitions at runtime

Interview Tip: `spark.sql.adaptive.skewJoin.enabled` is the modern, code-free fix for this.

R CONTROLLING PARTITION COUNT

Both change how many partitions you end up with — but they get there in very different ways.

Two Ways to Resize

.repartition(n)

Full shuffle. Can **increase or decrease** partitions. Produces evenly sized partitions — can fix skew.

.coalesce(n)

No full shuffle. Can only **decrease** partitions. Merges neighbors — sizes can stay uneven.

1 Use **repartition()** before a shuffle-heavy op, or to fix skew



2 Use **coalesce()** after filtering most rows out, right before writing output

Interview Tip: `coalesce()` is cheaper but can't fix skew — it just glues existing partitions together.

Save this slide before your next Data Engineering interview

CONCEPT	WHAT IT CONTROLS	KEY INTERVIEW FACT
Partition	Data distribution	One partition = one Task
Shuffle	Cross-partition movement	Disk + network, most expensive op
Skew	Uneven partition sizes	One Task straggles; fix via salting/AQE
repartition(n)	Full shuffle	Even sizes, can increase or decrease
coalesce(n)	No full shuffle	Only decreases, can stay uneven

Follow for daily Data Engineering content

@chintapradeep

Spark · Python · SQL · Interview Prep